AI & LLM Fundamentals — A Consultant's Guide

AI & LLM Fundamentals

A Consultant's Guide to Modern Artificial Intelligence

Everything you need to speak confidently about AI, LLMs, RAG, embeddings, and modern AI architecture — from first principles to production patterns.

Version 1.0 — June 2026

Prepared for: Technical conversations, client meetings, and interviews

Page 1

AI & LLM Fundamentals — A Consultant's Guide

Table of Contents

Page 2

AI & LLM Fundamentals — A Consultant's Guide

Page 3

AI & LLM Fundamentals — A Consultant's Guide

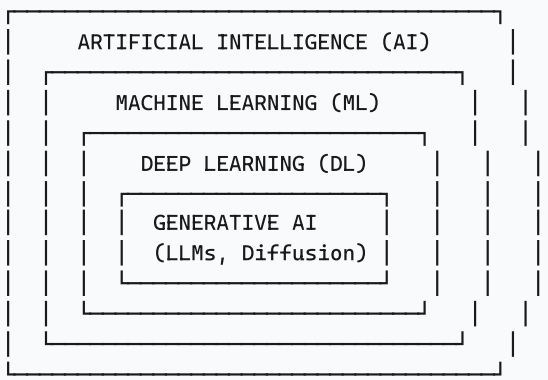
1. What is Artificial Intelligence?

AI is not magic. It's mathematics, data, and engineering — applied at scale.

1.1 AI vs. ML vs. DL vs. Generative AI

These terms form a nested hierarchy. Understanding the distinction is essential for any technical conversation:

Term	Definition	Example
Artificial Intelligence (AI)	The broadest field — any system that performs tasks requiring human-like intelligence.	A chess program, a spam filter, a self-driving car
Machine Learning (ML)	A subset of AI where systems learn patterns from data rather than following explicit rules.	Email spam classifier trained on labeled examples
Deep Learning (DL)	A subset of ML using multi-layered neural networks to learn hierarchical representations.	Image recognition with CNNs, language translation with transformers
Generative AI	A subset of DL that generates new content (text, images, code, audio) rather than just classifying or predicting.	ChatGPT, DALL-E, GitHub Copilot, Midjourney



1.2 The AI Spectrum: Narrow AI to AGI

- **Narrow AI (ANI):** Excels at one specific task. Everything we have today — ChatGPT, AlphaFold, self-driving cars — is narrow AI. "Narrow" doesn't mean limited; GPT-4 is narrow AI because it only does

Page 4

AI & LLM Fundamentals — A Consultant's Guide

language.

- **General AI (AGI):** A hypothetical system matching human-level performance across any intellectual task. Does not exist. Estimates range from 10 to 100+ years away.
- **Superintelligence (ASI):** Exceeds human capability in virtually all domains. Purely theoretical.

> **Interview tip:** When someone says "AGI is coming next year," ask them to define AGI precisely. You'll find no consensus definition exists. This is a sign of AI literacy.

1.3 The Current AI Landscape (2026)

Key trends shaping the industry right now:

- **Model commoditization:** Frontier models (GPT-4 class) are converging in capability. The moat is shifting from model quality to data, tooling, and application architecture.
- **Local/private inference:** Models like Llama 3.x, Qwen 2.5, Mistral, and DeepSeek run on consumer hardware. Enterprises increasingly demand on-premises AI.
- **Multimodal models:** Models that process text, images, audio, and video natively — not just text with vision bolted on.
- **Agentic AI:** Systems that plan, use tools, and execute multi-step tasks autonomously. Still unreliable for high-stakes decisions.
- **Regulation arriving:** EU AI Act is in effect. Other jurisdictions are following. "Move fast and break things" is being replaced by compliance frameworks.

Page 5

AI & LLM Fundamentals — A Consultant's Guide

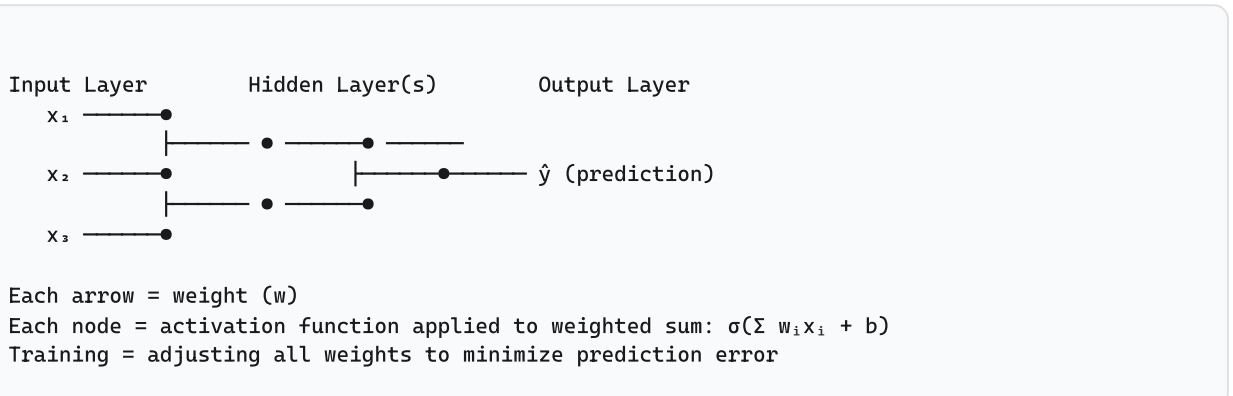
2. Machine Learning Fundamentals

2.1 Supervised vs. Unsupervised vs. Reinforcement Learning

Type	How It Works	Example	Data Required
Supervised Learning	Learn a mapping from inputs to outputs using labeled examples.	Classify emails as spam/not-spam from a dataset of labeled emails.	Labeled pairs: (input, correct_output)
Unsupervised Learning	Find patterns in data without labels.	Group customers into segments based on purchasing behavior.	Unlabeled data only
Reinforcement Learning	Learn by taking actions in an environment to maximize a reward signal.	Train an agent to play Go (AlphaGo).	Environment with rewards
Self-Supervised Learning	The model creates its own labels from the data structure (key to modern LLMs).	Mask words in a sentence and train the model to predict them (BERT, GPT pretraining).	Unlabeled text — the "label" is the next word

2.2 Neural Networks: The Building Block

A neural network is a function approximator inspired (loosely) by biological neurons:



Key concepts:

- **Weights (w):** The learnable parameters. A modern LLM has billions.
- **Activation function (σ):** Introduces non-linearity. Without it, stacking layers is equivalent to a single linear transform. Common: ReLU, GELU, SiLU.

Page 6

AI & LLM Fundamentals — A Consultant's Guide

- **Loss function:** Measures how wrong the prediction is. Training minimizes this.
- **Backpropagation:** The algorithm that computes how much each weight contributed to the error, enabling weight updates via gradient descent.

2.3 Training, Validation, and Testing

- **Training set (typically 70-80%):** Data the model learns from. Weights are updated based on this data.
- **Validation set (typically 10-15%):** Used to tune hyperparameters and detect overfitting. Not used for weight updates directly.
- **Test set (typically 10-15%):** Held out until the very end. Used exactly once to report final performance. Using it more than once invalidates its purpose.

> **Common pitfall:** Data leakage — when information from the test set leaks into training (e.g., normalizing using statistics from the entire dataset before splitting). This produces unrealistically optimistic results.

2.4 Key Metrics: Accuracy, Precision, Recall, F1

Metric	Formula	When to Use
Accuracy	(TP + TN) / Total	Balanced classes. Misleading for imbalanced data (e.g., 99% not-fraud).
Precision	TP / (TP + FP)	When false positives are costly. "Of what we flagged, how many were correct?"
Recall	TP / (TP + FN)	When false negatives are costly. "Of all actual positives, how many did we find?"
F1 Score	2 × (P × R) / (P + R)	Harmonic mean of precision and recall. Balances both concerns.

Page 7

AI & LLM Fundamentals — A Consultant's Guide

3. Large Language Models (LLMs)

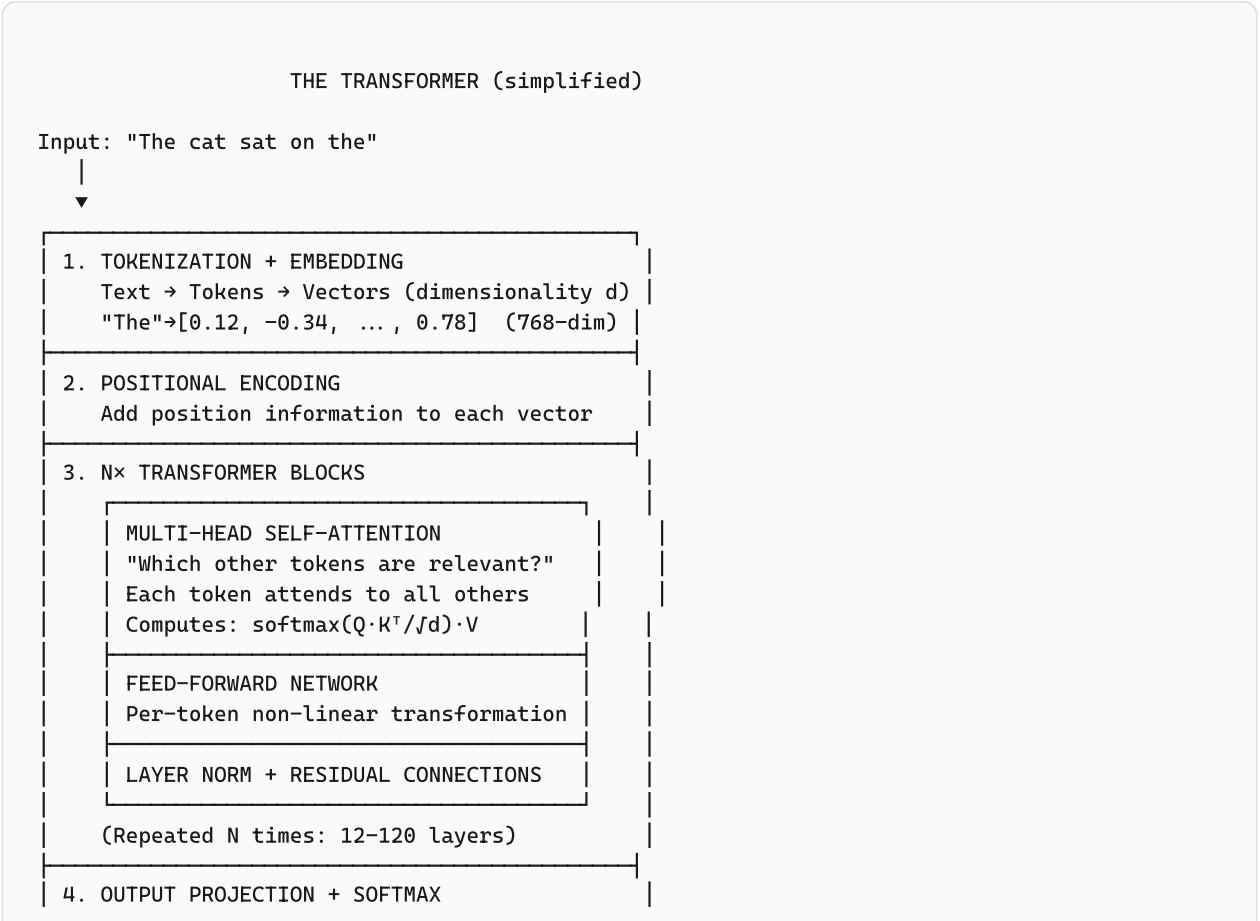
3.1 What is an LLM?

A **Large Language Model** is a neural network trained on vast amounts of text to predict the next token in a sequence. This deceptively simple objective — "predict the next word" — produces models that can translate, summarize, reason, code, and converse.

> **The core insight of LLMs:** If you train a sufficiently large model on sufficiently diverse text to predict the next token well enough, the model must develop internal representations of grammar, facts, reasoning patterns, and world knowledge — because that's what it takes to predict the next token accurately across billions of diverse contexts.

3.2 The Transformer Architecture

Introduced in the 2017 paper "*Attention Is All You Need*" (Vaswani et al.), the Transformer is the architecture behind virtually all modern LLMs:



Page 8

AI & LLM Fundamentals — A Consultant's Guide

```
│    → Probability distribution over vocabulary │
│    → Sample/pick next token: "mat"            │
└──────────────────────────────────────────────┘
```

The attention mechanism is the key innovation:

- Each token "attends to" every other token in the sequence.
- Attention scores determine how much each token influences the representation of every other token.
- **Multi-head attention** runs multiple attention operations in parallel, allowing the model to attend to different aspects of the input simultaneously (syntax, semantics, reference, etc.).
- The formula: $\text{Attention}(Q,K,V) = \text{softmax}(QK^T/\sqrt{d_k})V$ — where Q (query), K (key), and V (value) are learned projections of the input.

3.3 How LLMs Are Trained

Modern LLM training typically has three phases:

Phase	What Happens	Data Scale	Compute
1. Pretraining	Train on trillions of tokens from the internet, books, and code. Next-token prediction objective. This is where the model learns language, facts, and reasoning.	1–15 trillion tokens	Thousands of GPUs for weeks/months. Costs $10M–$100M+
2. Instruction Tuning (SFT)	Fine-tune on high-quality instruction-response pairs. Teaches the model to follow instructions rather than just complete text.	10K–1M examples	Dozens of GPUs for hours/days
3. RLHF / DPO	Align the model with human preferences. RLHF uses a reward model trained on human comparisons. DPO (Direct Preference Optimization) does this more efficiently without a separate reward model.	10K–100K preference pairs	Similar to SFT

> **Why this matters for consultants:** When a client says "can we train an LLM on our data?", they usually mean fine-tuning (phase 2), not pretraining (phase 1). Pretraining from scratch is economically infeasible for almost everyone. Clarify this distinction early.

3.4 Tokens, Context Windows, and Parameters

Concept	What It Is	Why It Matters

Page 9

AI & LLM Fundamentals — A Consultant's Guide

Token	The atomic unit of text for an LLM. ~0.75 English words. "The cat" → 2 tokens. Tokens are not characters and not exactly words — they're subword units from a tokenizer like BPE (Byte-Pair Encoding).	Pricing (per 1K tokens), context limits, and latency all revolve around tokens.
Context Window	Maximum number of tokens the model can process in one forward pass. Modern models: 4K (GPT-3.5), 128K (GPT-4 Turbo), 200K (Claude), 1M+ (Gemini).	Determines how much information you can provide in a single prompt. Long context doesn't mean the model *uses* all of it effectively (the "needle in a haystack" problem).
Parameters	The learnable weights in the neural network. 7B = 7 billion parameters, ~14GB at 16-bit precision. Range: 1B (tiny) to 1.7T+ (frontier).	More parameters ≈ more capacity, but also more VRAM and slower inference. A 7B model runs on a laptop; a 405B model needs a datacenter.

3.5 Major LLM Families (2026)

Family	Developer	Key Models	Open Weights?	Notable
GPT	OpenAI	GPT-4o, GPT-4.1, o3, o4-mini	No	Market leader. Multimodal. The "reasoning" o-series uses chain-of-thought at inference time.
Claude	Anthropic	Claude Opus 4.7, Sonnet 4.6, Haiku 4.5	No	Emphasis on safety and long-context reasoning. Constitutional AI training approach.
Gemini	Google DeepMind	Gemini 2.5 Pro/Flash	No	Natively multimodal (text, image, audio, video). Largest context windows (1M-2M tokens).
Llama	Meta	Llama 3.1 (8B/70B/405B), Llama 3.2 (multimodal)	Yes	Most widely used open-weight models. Foundation of most fine-tuned models and local deployments.
Qwen	Alibaba	Qwen 2.5 (0.5B–72B), Qwen 3	Yes	Strong multilingual performance. Excellent for non-English use cases. Competitive with Llama 3.
Mistral	Mistral AI	Mistral Large, Mixtral 8x22B, Codestral	Some	European champion. Mixture-of-Experts architecture. Strong code generation.

Page 10

AI & LLM Fundamentals — A Consultant's Guide

DeepSeek	DeepSeek	DeepSeek-V3, DeepSeek-R1	Yes	Extremely cost-efficient training. R1 uses reinforcement-learning-based reasoning.

Page 11

AI & LLM Fundamentals — A Consultant's Guide

4. Embeddings & Vector Representations

4.1 What Are Embeddings?

An **embedding** is a dense vector (list of floating-point numbers) that represents the semantic meaning of a piece of text in a high-dimensional space. Texts with similar meanings have vectors that are close together.

```
"The king ruled the kingdom"   →  [0.12, -0.34, 0.78, ..., 0.05]  (768 dimensions)
"The queen governed the realm" →  [0.14, -0.31, 0.75, ..., 0.03]  (very close!)
"Banana smoothie recipe"       →  [-0.56, 0.82, -0.13, ..., 0.67]  (far away!)
```

Embeddings are the bridge between human language and machine computation. They turn "meaning" into math — specifically, into vectors where distance equals semantic similarity.

4.2 How Embeddings Are Generated

Embedding models are specialized neural networks trained to map text to vectors:

- **Encoder-only models:** BERT, RoBERTa, and their descendants. Designed to produce high-quality sentence/paragraph embeddings.
- **Decoder-only models:** GPT-style models can be used for embeddings by taking the hidden state at the last token, but they are not optimized for this.
- **Contrastive training:** Modern embedding models (e.g., `nomic-embed-text`, `bge-large`, `text-embedding-3`) are trained to maximize similarity for related texts and minimize it for unrelated texts — often using billions of text pairs.

4.3 Cosine Similarity & Vector Search

The most common way to compare embeddings:

```
cosine_similarity(A, B) = (A · B) / (|A| × |B|)
```

Properties:

- Range: -1 to +1 (for normalized vectors, 0 to 1 in practice).
- 1.0 = identical direction (semantically identical).
- 0.0 = orthogonal (unrelated).
- -1.0 = opposite direction (rare in practice with modern embeddings).

Page 12

AI & LLM Fundamentals — A Consultant's Guide

Vector search in a database (Qdrant, Pinecone, pgvector) works by:

1. Embedding the query text into a vector.
2. Finding the K vectors in the database with highest cosine similarity (or lowest Euclidean distance) to the query vector.
3. Returning the corresponding documents/chunks.

For efficiency, vector databases use **Approximate Nearest Neighbor (ANN)** algorithms (HNSW, IVF) rather than exact exhaustive search over millions of vectors.

4.4 Popular Embedding Models

Model	Dimensions	Max Tokens	Open?	Best For
`nomic-embed-text`	768	8192	Yes (Ollama)	General purpose, local deployment, English
`text-embedding-3-small`	512/1536	8191	No (OpenAI API)	Cost-efficient cloud embeddings
`text-embedding-3-large`	256/1024/3072	8191	No (OpenAI API)	Highest quality cloud embeddings
`bge-large-en-v1.5`	1024	512	Yes (HuggingFace)	High quality open-source English embeddings
`mxbai-embed-large`	1024	512	Yes (Ollama)	Strong open-source alternative
`multilingual-e5-large`	1024	512	Yes (HuggingFace)	Multi-language (100+ languages)

Page 13

AI & LLM Fundamentals — A Consultant's Guide

5. Retrieval-Augmented Generation (RAG)

5.1 Why RAG? The Hallucination Problem

LLMs have a fundamental limitation: their knowledge is frozen at training time. They don't know about your documents, your company's policies, or last week's news. Worse, when asked about things they don't know, they often **hallucinate** — generating plausible-sounding but factually incorrect information.

RAG solves this by giving the LLM access to relevant information at query time.

Without RAG

```
User: "What is our
return policy?"

LLM: "Most companies offer
a 30-day return window ... "

(Generic, possibly wrong
for this specific company)
```

With RAG

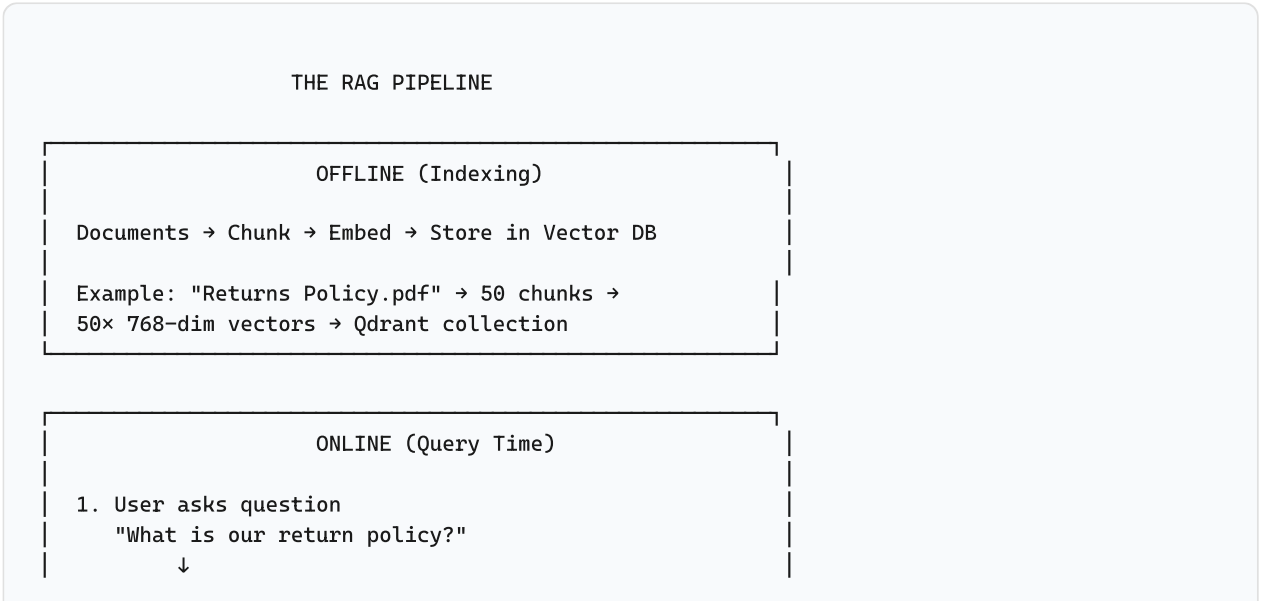
```
User: "What is our
return policy?"

System: [Retrieves actual
return policy document]

LLM: "Per the Returns Policy
document (v3, §2.1): Returns
are accepted within 60 days
with original receipt ... "

(Specific, sourced, correct)
```

5.2 The RAG Architecture



Page 14

AI & LLM Fundamentals — A Consultant's Guide

```
│  2. Embed the question → query vector [768-dim]          │
│          ↓                                               │
│  3. Vector search → top-K relevant chunks                │
│          ↓                                               │
│  4. Construct prompt:                                    │
│     "Answer using ONLY these sources:                    │
│      [Source 1] Returns Policy §2.1: ...                 │
│      [Source 2] Returns Policy §3.4: ...                 │
│      Question: What is our return policy?"               │
│          ↓                                               │
│  5. LLM generates answer grounded in sources             │
│          ↓                                               │
│  6. Return answer with source citations                  │
└──────────────────────────────────────────────────────────┘
```

5.3 Advanced RAG Patterns

Basic RAG is just the starting point. Production systems use more sophisticated patterns:

Pattern	How It Works	When to Use
Hybrid Search RAG	Combine keyword (BM25) + vector search for retrieval. Fuse results with weighted scoring.	When you need both exact term matching and semantic understanding. (Used in our platform.)
Reranked RAG	Retrieve N candidates, then apply a cross-encoder reranker to score relevance more accurately before sending top-K to the LLM.	When retrieval quality is critical and the added latency is acceptable.
Agentic RAG	The LLM decides whether to search, what to search for, and iterates — refining queries based on intermediate results.	Complex multi-hop questions: "Compare the liability clauses across all contracts signed in Q3."
Graph RAG	Augment vector retrieval with knowledge graph traversal. Entities and relationships provide structured context alongside unstructured text chunks.	Legal, medical, or scientific domains with rich structured knowledge bases. (Used in our platform via Neo4j.)
Small-to-Big Retrieval	Index small chunks for precise retrieval, but feed larger surrounding context to the LLM for generation.	When you want retrieval precision but generation needs broader context.

5.4 RAG vs. Fine-Tuning vs. Long Context

These three approaches solve different problems, and the right choice depends on the use case:

Approach	What It Does	Best For	Limitations

Page 15

AI & LLM Fundamentals — A Consultant's Guide

RAG	Retrieves relevant info at query time. Knowledge is external, updatable, and auditable.	Factual Q&A over documents, policy lookup, compliance checking. Dynamic/updating knowledge bases.	Retrieval quality is the bottleneck. Garbage retrieval = garbage generation. Adds latency.
Fine-Tuning	Continues training the model on domain-specific data. Changes how the model behaves, not just what it knows.	Teaching a specific style, format, or reasoning pattern. Domain terminology adaptation.	Knowledge still becomes stale. Expensive to update. Can cause catastrophic forgetting of general capabilities.
Long Context	Put all relevant documents directly in the prompt (up to the context window limit).	Small document sets, one-off analysis tasks.	Cost scales quadratically with context length (attention is $O(n^2)$). Models don't attend equally to all parts of long contexts. Latency increases with context size.

> **Best practice:** RAG and fine-tuning are complementary, not competing. Use fine-tuning to teach the model *how* to answer (format, style, reasoning approach) and RAG to provide *what* to answer from (up-to-date facts from your documents).

Page 16

AI & LLM Fundamentals — A Consultant's Guide

6. Prompt Engineering

6.1 What is Prompt Engineering?

Prompt engineering is the practice of designing inputs to LLMs that elicit desired outputs. It's part science (understanding how models process prompts), part craft (iterative refinement), and part engineering (building reliable, versioned prompt templates for production systems).

6.2 Core Techniques

Technique	Description	Example
Zero-shot	Ask directly with no examples.	"Classify this email as spam or not spam: [email]"
Few-shot	Provide 2–5 examples in the prompt.	"Here are examples: Email A → Spam. Email B → Not spam. Now classify: [email]"
Chain of Thought (CoT)	Ask the model to reason step by step before answering.	"Let's think through this step by step. First, identify the key facts. Then..."
Role Prompting	Assign a persona to shape the response.	"You are a senior corporate attorney reviewing this contract..."
Structured Output	Specify the exact format you want.	"Respond in JSON with fields: 'answer', 'confidence', 'sources'."
Negative Prompting	Explicitly state what NOT to do.	"Do not speculate. If you don't know, say so. Do not invent citations."
Context Grounding	Provide source material and instruct the model to only use it.	"Answer using ONLY the following sources. Cite the source number for each claim."

6.3 Prompt Templates & Versioning

In production, prompts are not handwritten per-query — they are templates:

```
// Prompt template with version tracking
templateVersion: "v2.3-legal-analysis"
systemPrompt: "You are a legal document analyst. Answer using ONLY
               the provided sources. Cite [Source N] for each claim.
               If sources are insufficient, say so explicitly.
               Do not invent legal citations."
```


Page 17

AI & LLM Fundamentals — A Consultant's Guide

```
userPrompt: "Sources:
             {{#each sources}}
             [Source {{@index}}] {{this.text}} ({{this.title}}, §{{this.section}})
             {{/each}}

             Question: {{question}}

             Provide a structured answer with:
             1. Direct Answer
             2. Supporting Evidence (with source citations)
             3. Limitations (what the sources don't address)
             4. Confidence Assessment (High/Medium/Low)"
```

> **Why version prompts:** When an AI answer is used in a legal or compliance context, you must be able to reproduce it. Prompt template version, model version, and retrieval sources must all be recorded. (Our platform does this via the audit system.)

Page 18

AI & LLM Fundamentals — A Consultant's Guide

7. AI System Architecture Patterns

Building a demo is easy. Building a production AI system that is reliable, auditable, and maintainable requires deliberate architecture.

7.1 The Modular Monolith Pattern

While microservices dominate conference talks, the **modular monolith** is often the right choice for AI systems:

- **Transactional integrity:** Document upload → text extraction → chunking → embedding → dual indexing should be one atomic operation. Distributed transactions (sagas) add complexity that provides no business value here.
- **Deployment simplicity:** `java -jar app.jar` vs. orchestrating 6+ microservices.
- **Refactoring safety:** Compile-time boundaries between modules prevent circular dependencies. You can extract a module into a microservice later if needed.
- **Observability:** One process to monitor, one log stream, one correlation ID space.

7.2 Provider Abstraction

The most important architectural pattern in AI systems: **always abstract the AI provider behind an interface**.

```
// Interface (in api/ package)
public interface ChatCompletionProvider {
    String complete(String prompt, ModelCapabilities capabilities);
}

// Implementations (in application/ package)
public class OllamaChatCompletionProvider implements ChatCompletionProvider { ... }
public class OpenAiChatCompletionProvider implements ChatCompletionProvider { ... }
public class AnthropicChatCompletionProvider implements ChatCompletionProvider { ... }
```

Benefits:

- **No vendor lock-in:** Switch from Ollama to OpenAI by changing one Spring bean.
- **Testability:** Unit tests use a mock implementation. No network calls in tests.
- **Graceful degradation:** The `NoOpChatCompletionProvider` returns a clear "not configured" message instead of crashing.
- **A/B testing:** Run multiple implementations side by side for comparison.

Page 19

AI & LLM Fundamentals — A Consultant's Guide

7.3 Grounding & Citation

Ungrounded AI is a liability. The grounding pattern ensures every AI claim can be traced to evidence:

1. **Retrieve** relevant source passages (chunks) for the query.
2. **Include** those passages in the prompt with identifiers ([Source 1], [Source 2]).
3. **Instruct** the LLM to cite sources inline.
4. **Post-process** the answer to extract and validate citations against the retrieval context.
5. **Flag** uncited claims for review.

Our platform's `GroundingService` and `ClaimValidator` implement this pattern as a post-inference quality gate.

7.4 Guardrails & Validation

Production AI systems need multiple layers of validation:

Layer	What It Checks	When It Runs
Input validation	Is the question non-empty, safe, within length limits?	Before any processing
Intent classification	Should this go to RAG, index inspection, or be rejected?	Before retrieval
Retrieval quality	Are there enough relevant sources? Are scores above threshold?	After retrieval, before LLM call
Temporal consistency	Does the answer respect chronological constraints?	After LLM call
Claim validation	Are factual claims supported by retrieved sources?	After LLM call
Output formatting	Is the response well-structured? Does it include citations?	Before returning to user

Page 20

AI & LLM Fundamentals — A Consultant's Guide

8. Running LLMs Locally

8.1 Why Local Inference?

Five reasons enterprises choose local over API-based LLMs:

1. **Data sovereignty:** Documents (legal, medical, classified) never leave the premises.
2. **Cost at scale:** At 10K+ queries/day, API costs exceed hardware costs within months.
3. **Latency:** No network round-trip. Sub-50ms response times achievable.
4. **Availability:** No dependency on external service uptime. Works air-gapped.
5. **Compliance:** Satisfies GDPR, HIPAA, ITAR, and other data residency requirements.

8.2 Ollama, vLLM, llama.cpp

Tool	Type	Best For	Key Feature
Ollama	Desktop/server LLM runner	Development, small-scale production, ease of use	Simple CLI, REST API, model library, GPU auto-detection. `ollama run qwen2.5:14b`.
llama.cpp	C++ inference engine	CPU inference, edge devices, maximum portability	Runs on CPU (no GPU needed). GGUF quantized model format. Powers most local LLM tools under the hood.
vLLM	High-throughput serving	Production serving at scale, multi-user	PagedAttention for efficient KV cache management. OpenAI-compatible API. 10-20× higher throughput than naive serving.
LM Studio	Desktop GUI	Non-technical users, exploration	Point-and-click model download and chat. Built on llama.cpp.

8.3 Hardware Considerations

Model Size	VRAM Required (4-bit quantized)	Example Hardware
1B–3B parameters	1–2 GB	Any modern laptop, Raspberry Pi 5 (8GB)
7B–8B parameters	4–6 GB	Laptop with 8GB VRAM (RTX 3070, M2 MacBook)

Page 21

AI & LLM Fundamentals — A Consultant's Guide

13B–14B parameters	8–10 GB	RTX 3080/4070 (12GB), M3 Max MacBook (36GB unified)
30B–34B parameters	18–22 GB	RTX 4090 (24GB), M2 Ultra (76GB unified)
70B–72B parameters	40–48 GB	2× RTX 4090, A100 (80GB), Mac Studio (192GB unified)
405B parameters	200+ GB	4× A100/H100, datacenter GPU cluster

> **Key insight:** Quantization (4-bit, 8-bit) reduces model size by 4–8× with minimal quality loss. A 7B model at 4-bit uses ~4GB VRAM and runs on a laptop. This is what makes local AI economically viable.

Page 22

AI & LLM Fundamentals — A Consultant's Guide

9. AI Ethics, Safety & Governance

9.1 Hallucination & Factual Accuracy

Hallucination is when an LLM generates text that is grammatically correct and semantically plausible but factually wrong. It's not a bug — it's a property of how these models work (they predict likely tokens, not true facts).

Mitigation strategies (layered):

1. **RAG:** Ground answers in retrieved documents.
2. **Citation requirements:** Require the model to cite sources for factual claims.
3. **Post-hoc validation:** Check claims against knowledge bases.
4. **Confidence calibration:** Show users how confident the system is in each claim.
5. **Human review:** For high-stakes outputs, a human must verify before action.

9.2 Bias, Fairness, and Representation

LLMs inherit biases from their training data — which is predominantly English-language internet text produced by a narrow demographic slice of humanity. Key concerns:

- **Representation bias:** Models perform worse for underrepresented languages, dialects, and cultural contexts.
- **Stereotype reinforcement:** Models may reproduce gender, racial, and occupational stereotypes present in training data.
- **Access inequality:** Frontier models are expensive and controlled by a few corporations. Open-weight models (Llama, Qwen) are partially addressing this.

> **Consultant's framing:** Don't claim to "solve" bias — it's a research problem. Instead, describe the *mitigations* you implement: diverse evaluation sets, output monitoring, user feedback loops, and transparency about model limitations.

9.3 Data Privacy & Sovereignty

When building AI systems, data flows matter:

- **API-based models (OpenAI, Anthropic):** Your data is sent to a third-party server. Check the data usage policy — some providers use API data for training (OpenAI no longer does for API calls; always verify current policy).

Page 23

AI & LLM Fundamentals — A Consultant's Guide

- **Local models (Ollama, vLLM):** Data never leaves your infrastructure. This is the only acceptable option for regulated industries.
- **RAG considerations:** Even with local inference, retrieval may pull sensitive documents into the prompt. Access controls must apply at the retrieval stage, not just the LLM stage.

9.4 The EU AI Act & Regulatory Landscape

The EU AI Act (effective 2024–2026, phased implementation) is the world's first comprehensive AI regulation. Key points:

Risk Level	Examples	Requirements
Unacceptable Risk	Social scoring, real-time biometric surveillance in public spaces, manipulative AI	Prohibited entirely
High Risk	AI in hiring, credit scoring, medical diagnosis, legal document analysis, law enforcement	Conformity assessment, human oversight, transparency, accuracy, robustness, record-keeping
Limited Risk	Chatbots, emotion recognition, deepfake generation	Transparency obligations (users must know they're interacting with AI)
Minimal Risk	AI in video games, spam filters, inventory management	No specific requirements

> **Document intelligence and legal AI is likely high-risk** under the EU AI Act if used for legal decision support. This means your platform needs audit trails, accuracy metrics, human oversight mechanisms, and transparency documentation. Our platform's audit system, grounding, and confidence scoring are direct responses to these regulatory requirements.

Page 24

AI & LLM Fundamentals — A Consultant's Guide

10. Key Terminology Cheat Sheet

Term	Definition
Attention	Mechanism that lets each token "look at" every other token. The core innovation of transformers.
Autoregressive	A model that generates one token at a time, conditioning on previously generated tokens. GPT-style models are autoregressive.
Chunking	Splitting documents into smaller pieces for embedding and retrieval. Critical parameter: chunk size and overlap.
Context Window	Max tokens the model can process in one pass. Ranges from 4K to 2M+ tokens in 2026.
Cross-Encoder	A model that scores a query-document pair together (vs. a bi-encoder that encodes them separately). More accurate but slower.
Embedding	A dense vector representation of text, where semantic similarity ≈ vector proximity.
Few-Shot Learning	Providing examples in the prompt to guide model behavior without weight updates.
Fine-Tuning	Continuing training of a pretrained model on domain-specific data to adapt its behavior or knowledge.
GGUF	File format for quantized models used by llama.cpp and Ollama. Enables running large models on consumer hardware.
Grounding	Linking AI-generated claims back to specific source documents or passages.
HNSW	Hierarchical Navigable Small World — an ANN algorithm used by Qdrant, Weaviate, and others for fast vector search.
Latency	Time from request to first token (TTFT) and total time to completion. Critical UX metric for AI features.
LoRA / QLoRA	Parameter-efficient fine-tuning methods. Train only a small adapter (1–10% of parameters) rather than the full model.
Mixture of Experts (MoE)	Architecture where only a subset of parameters is active for any given input. Mixtral, DeepSeek-V3. More capacity at lower inference cost.
Quantization	Reducing numerical precision of model weights (FP16 → INT8 → INT4). Reduces VRAM by 2–8× with minimal quality loss.
RAG	Retrieval-Augmented Generation — giving an LLM access to external knowledge at query time via retrieval.

Page 25

AI & LLM Fundamentals — A Consultant's Guide

RLHF	Reinforcement Learning from Human Feedback — aligning model outputs with human preferences using a learned reward model.
Temperature	Parameter controlling randomness in token sampling. 0 = deterministic, 1 = creative, >1 = increasingly random.
Token	Atomic unit of LLM input/output. ~0.75 words in English. "Artificial intelligence" → ~3 tokens.
Transformer	The neural network architecture introduced in 2017 that powers virtually all modern LLMs.
Vector Database	Database optimized for storing and searching vector embeddings. Qdrant, Pinecone, pgvector, Weaviate, Milvus.
Zero-Shot	Asking a model to perform a task with no examples, relying entirely on its pretraining and instruction tuning.

Page 26

AI & LLM Fundamentals — A Consultant's Guide

11. Conversation-Ready Talking Points

When the conversation turns to AI in an interview or client meeting, here are concise positions you can take:

"Should we use a cloud API or run models locally?"

"It depends on three factors: data sensitivity, query volume, and latency requirements. For regulated industries or high volumes, local models (Ollama, vLLM) with open-weight models like Llama 3 or Qwen 2.5 give you data sovereignty, predictable costs, and lower latency. For prototyping or low-volume use, cloud APIs are faster to start. A well-architected system abstracts the provider so you can switch without rewriting application logic."

"RAG vs. fine-tuning — which should we use?"

"They solve different problems. RAG gives the model access to up-to-date facts from your knowledge base at query time. Fine-tuning changes how the model behaves — its style, reasoning patterns, and domain fluency. For most enterprise use cases, start with RAG for knowledge and add fine-tuning for behavior. They're complementary, not competing. And long context windows aren't a replacement for RAG — attention is $O(n^2)$ and models don't use all parts of long contexts equally well."

"How do we prevent the AI from making things up?"

"You can't eliminate hallucination entirely — it's inherent to how these models work. But you can build a system that makes hallucinations *detectable* and *manageable*. The approach we use is layered: (1) RAG to ground answers in real documents, (2) prompt instructions to cite sources and refuse to speculate, (3) post-inference validation that checks claims against the retrieval context, (4) confidence scoring so users know when to trust the output, and (5) audit trails so answers can be reviewed after the fact."

"What model should we use?"

Page 27

AI & LLM Fundamentals — A Consultant's Guide

"The model itself matters less than the architecture around it. GPT-4, Claude, Llama 3, Qwen 2.5 — the frontier models are converging in capability. The real differentiation is in your retrieval quality, your prompt engineering, your validation pipeline, and your data. Choose a model that meets your latency and cost requirements, then invest in the system around it. And always abstract the provider so you can swap models as the landscape evolves."

"Is AI going to replace [X profession]?"

"AI automates tasks, not professions. A lawyer who uses AI to find relevant precedents in seconds instead of hours is still a lawyer — just a more effective one. The professionals who will be 'replaced' are those who refuse to use these tools. The professionals who adopt AI as an augmentation tool will be more valuable, not less. Our platform is designed for that augmentation model — the AI proposes answers with citations, the human professional reviews and decides."

"How do you handle the EU AI Act compliance?"

"For high-risk AI systems, the EU AI Act requires transparency, human oversight, accuracy metrics, and record-keeping. Our platform addresses this through: comprehensive audit logging of every AI interaction, source citations for every claim, confidence scores, temporal validation, and a human-review-before-action workflow. The key principle is that the AI advises, the human decides, and everything is traceable."

Page 28

AI & LLM Fundamentals — A Consultant's Guide

12. Recommended Learning Path

If you want to go deeper, here's a structured path from fundamentals to production:

Foundations (Understand the basics)

1. Andrej Karpathy's "Intro to Large Language Models" (YouTube, 1 hour) — the best single introduction to LLMs.
2. Jay Alammar's "The Illustrated Transformer" (blog post) — visual explanation of the attention mechanism.
3. 3Blue1Brown's "Neural Networks" series (YouTube) — intuitive math of neural networks.
4. "Attention Is All You Need" (Vaswani et al., 2017) — the original transformer paper. Read it even if you don't understand everything.

Applied (Build things)

1. Set up Ollama locally. Run `ollama run qwen2.5:14b` and experiment with prompts.
2. Build a simple RAG system: chunk documents, embed with `nomic-embed-text`, store in Qdrant, query with an LLM.
3. Read the LangChain / LlamaIndex documentation — not necessarily to use them, but to understand the abstractions they provide.
4. Study the RAG survey papers: "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks" (Lewis et al., 2020) and more recent surveys.

Production (Engineer for reliability)

1. Study production RAG architectures: query classification, hybrid search, reranking, grounding, validation.
2. Learn about embeddings: contrastive learning, Matryoshka embeddings, multilingual embeddings.
3. Understand evaluation: RAGAS framework, LLM-as-judge evaluation, human evaluation protocols.
4. Read about AI safety and governance: Anthropic's research, EU AI Act text, NIST AI Risk Management Framework.

Stay Current

- Follow AI researchers and engineers on X/Twitter and LinkedIn.
- Read the arXiv CS.CL and CS.AI sections (or subscribe to summaries like "The Batch" by Andrew Ng).
- Watch conference talks: NeurIPS, ICML, ICLR, ACL (the papers are dense; the talks are accessible).

Page 29

AI & LLM Fundamentals — A Consultant's Guide

Final advice for the interview: You don't need to know everything. You need to know the fundamentals deeply, understand the tradeoffs in production AI systems, and be able to have an informed opinion grounded in experience. This guide gives you the conceptual framework. The Document Intelligence Platform gives you the concrete implementation. Together, they demonstrate that you can both *think* about AI and *build* AI — which is exactly what separates an AI consultant from someone who has just read about AI.

Page 30